

PYTHON - TKinter

PYTHON - Tkinter

Python provides the standard library Tkinter for creating the graphical user interface for desktop based applications.

An empty Tkinter top-level window can be created by using the following steps.

1. import the Tkinter module.
2. Create the main application window.
3. Add the widgets like labels, buttons, frames, etc. to the window.
4. Call the main event loop so that the actions can take place on the user's computer screen.

Example

Import tkinter

#creating the application main window.

top = tkinter.Tk()

#Entering the event main loop

top.mainloop()



Example

from tkinter **import** * or

#creating the application main window.

top = Tk()

#Entering the event main loop

top.mainloop()

Example

Import tkinter as tk

#creating the application main window.

top = tkinter.Tk()

#Entering the event main loop

top.mainloop()

Tkinter Widgets

Tkinter provides various controls, such as buttons, labels and text boxes used in a GUI application. These controls are commonly called widgets.

Widget	Description
<u>Label</u>	A label is a text used to display some message or information about the other widgets.
<u>Entry</u>	The entry widget is used to display the single-line text field to the user. It is commonly used to accept user values.
<u>Button</u>	The Button is used to add various kinds of buttons to the python application.
<u>Canvas</u>	The canvas widget is used to draw the canvas on the window.
<u>Text</u>	It is different from Entry because it provides a multi-line text field to the user so that the user can write the text and edit the text inside it.
<u>MessageBox</u>	This module is used to display the message-box in the desktop based applications.
<u>Frame</u>	It can be defined as a container to which, another widget can be added and organized.
<u>Menu</u>	It is used to add menu items to the user.
<u>Checkbutton</u>	The Checkbutton is used to display the CheckButton on the window.

Tkinter Widgets

Widget	Description
<u>Radiobutton</u>	The Radiobutton is different from a checkbutton. Here, the user is provided with various options and the user can select only one option among them.
<u>ListBox</u>	The ListBox widget is used to display a list of options to the user.
<u>Menubutton</u>	The Menubutton is used to display the menu items to the user.
<u>Message</u>	The Message widget is used to display the message-box to the user.
<u>Spinbox</u>	It is an entry widget used to select from options of values.

Label Widgets

This widget is used to provide the message to the user about other widgets used in the python application.

Syntax

```
w = Label (master, options)
```

Option	Description
bg	The background color displayed behind the widget.
bd	It represents the width of the border. The default is 2 pixels.
font	The font type of the text written inside the widget.
fg	The foreground color of the text written inside the widget.
height	The height of the widget.
text	This is set to the string variable which may contain one or more line of text.
textvariable	The text written inside the widget is set to the control variable StringVar so that it can be accessed and changed accordingly.
width	The width of the widget. It is specified as the number of characters.

```
from tkinter import *  
top=Tk()
```

```
top.geometry("400x250")
```

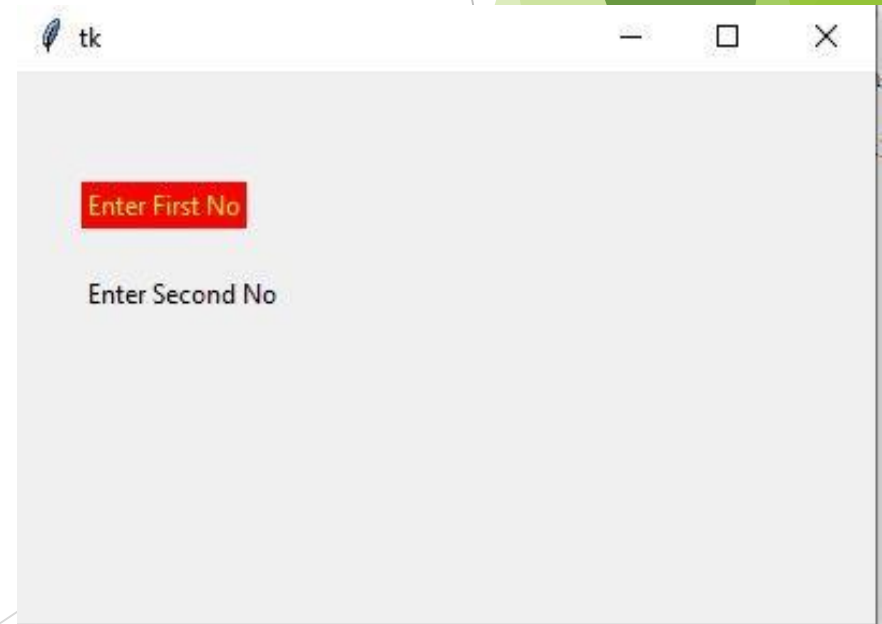
```
#creating label
```

```
Label(top,text="Enter First No",bg="red",fg="yellow").place(x=30,y=50)
```

```
#creating label
```

```
Label(top,text="Enter Second No").place(x=30,y=90)
```

```
top.mainloop()
```



Entry Widget

- The Entry widget is used to provide the single line text-box to the user to accept a value from the user.

Syntax

`w = Entry (parent, options)`

Option	Description
bg	The background color displayed behind the widget.
bd	It represents the width of the border. The default is 2 pixels.
font	The font type of the text written inside the widget.
fg	The foreground color of the text written inside the widget.
height	The height of the widget.
text	This is set to the string variable which may contain one or more line of text.
textvariable	It is set to the instance of the StringVar to retrieve the text from the entry.
width	The width of the widget. It is specified as the number of characters.

```
from tkinter import *
```

```
top=Tk()
```

```
top.geometry("400x250")
```

```
#creating label
```

```
Label(top,text="Enter First No",bg="red",fg="yellow").place(x=30,y=50)
```

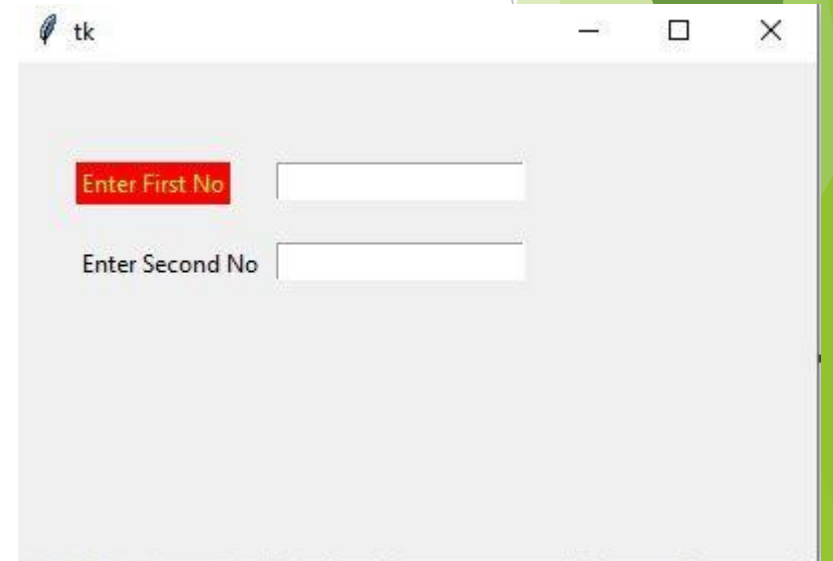
```
#creating label
```

```
Label(top,text="Enter Second No").place(x=30,y=90)
```

```
e1=Entry(top).place(x=130,y=50)
```

```
e2=Entry(top).place(x=130,y=90)
```

```
top.mainloop()
```



Button Widget

- ▶ The button widget is used to add various types of buttons to the python application.

Syntax

```
W = Button(parent, options)
```

Option	Description
bg	The background color displayed behind the widget.
bd	It represents the width of the border. The default is 2 pixels.
font	The font type of the text written inside the widget.
fg	The foreground color of the text written inside the widget.
height	The height of the widget.
text	This is set to the string variable which may contain one or more line of text.
width	The width of the widget. It is specified as the number of characters.
activebackground	It represents the background of the button when the mouse hover the button.
activeforeground	It represents the font color of the button when the mouse hover the button.
Command	It is set to the function call which is scheduled when the function is called.

```
from tkinter import *
```

```
top=Tk()
```

```
top.geometry("400x250")
```

```
#creating label
```

```
Label(top,text="Enter First No",bg="red",fg="yellow").place(x=30,y=50)
```

```
#creating label
```

```
Label(top,text="Enter Second No").place(x=30,y=90)
```

```
e1=Entry(top).place(x=130,y=50)
```

```
e2=Entry(top).place(x=130,y=90)
```

```
Button(top,text = "Press me",activebackgroun="yellow",activeforeground="red").place(x=130,y=140)
```

```
top.mainloop()
```



messagebox Widget

- ▶ The messagebox module is used to display the message boxes in the python applications.

Syntax

```
messagebox.function_name(title, message [,options])
```

Parameters

function_name	:	It represents an appropriate message box function.
title	:	It is a string which is shown as a title of a message box.
message	:	It is the string to be displayed as a message on the message box.
options	:	There are various options which can be used to configure the message dialog box.

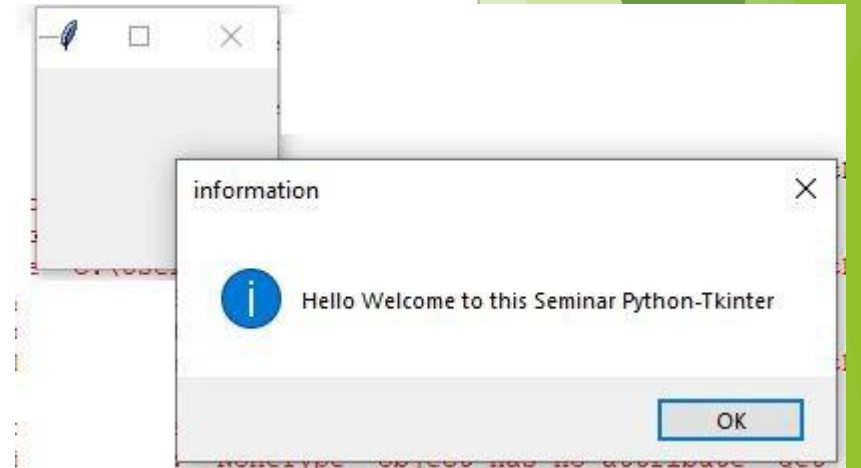
```
from tkinter import *  
from tkinter import messagebox
```

```
top=Tk()
```

```
top.geometry("100x100")
```

```
messagebox.showinfo("information","Hello Welcome to this Seminar Python-Tkinter")
```

```
top.mainloop()
```



Addition of Two Numbers and Display result in Label box

```
from tkinter import *  
from tkinter import messagebox  
  
def add():  
    c=int(e1.get())+int(e2.get())  
    messagebox.showinfo("Result",c)  
  
top=Tk()  
top.geometry("400x250")
```

```
Label(top,text="Enter First No",bg="red",fg="yellow").place(x=30,y=50)
```

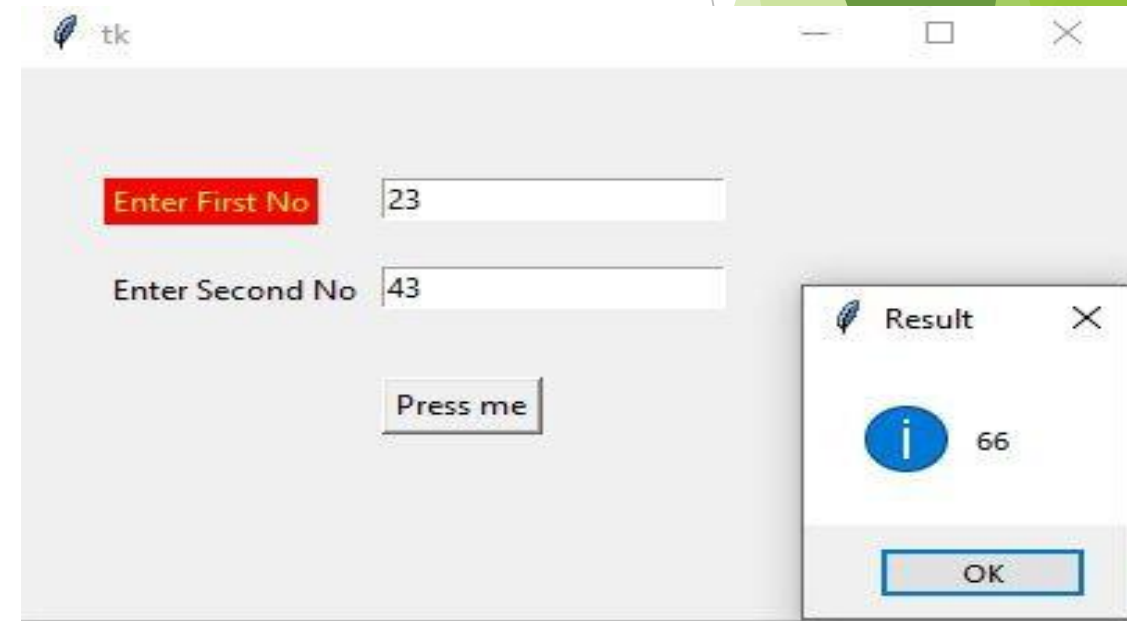
```
Label(top,text="Enter Second No").place(x=30,y=90)
```

```
e1=Entry(top)  
e1.place(x=130,y=50)
```

```
e2=Entry(top)  
e2.place(x=130,y=90)
```

```
Button(top,text = "Press me", command=add).place(x=130,y=140)
```

```
top.mainloop()
```



Grid in Tkinter

	columns		
rows	0,0	0,1	0,2
	1,0	1,1	1,2
	2,0	2,1	2,2
	3,0	3,1	3,2

<label 1>	<entry 2>
<label 1>	<entry 2>
<button 1>	

```
Label(top,text="Enter First No").grid(row=0,column=0)
```

```
Label(top,text="Enter Second No").grid(row=1,column=0)
```

```
e1=Entry(top)
```

```
e1.grid(row=0,column=1)
```

```
e2=Entry(top)
```

```
e2.grid(row=1,column=1)
```

```
Button(top,text = "Press me",command=add).grid(row=2,column=0)
```

Addition of Two Numbers Using Grid

```
from tkinter import *
```

```
from tkinter import messagebox
```

```
def add():
```

```
    c=int(e1.get())+int(e2.get())
```

```
    messagebox.showinfo("Result",c)
```

```
top=Tk()
```

```
top.geometry("400x250")
```

```
Label(top,text="Enter First No").grid(row=0,column=0)
```

```
Label(top,text="Enter Second No").grid(row=1,column=0)
```

```
e1=Entry(top)
```

```
e1.grid(row=0,column=1)
```

```
e2=Entry(top)
```

```
e2.grid(row=1,column=1)
```

```
Button(top,text = "Press me", command=add) .grid(row=2,column=0)
```

```
top.mainloop()
```



Addition of Two Numbers and Display result in Label Box

```
from tkinter import *  
from tkinter import messagebox  
def add():
```

```
    c=int(e1.get())+int(e2.get())
```

```
    res.set(c)
```

```
top=Tk()  
top.geometry("400x250")
```

```
res=StringVar()
```

```
Label(top,text="Enter First No").grid(row=0,column=0)
```

```
Label(top,text="Enter Second No").grid(row=1,column=0)
```

```
Label(top,text="Result").grid(row=2,column=0)
```

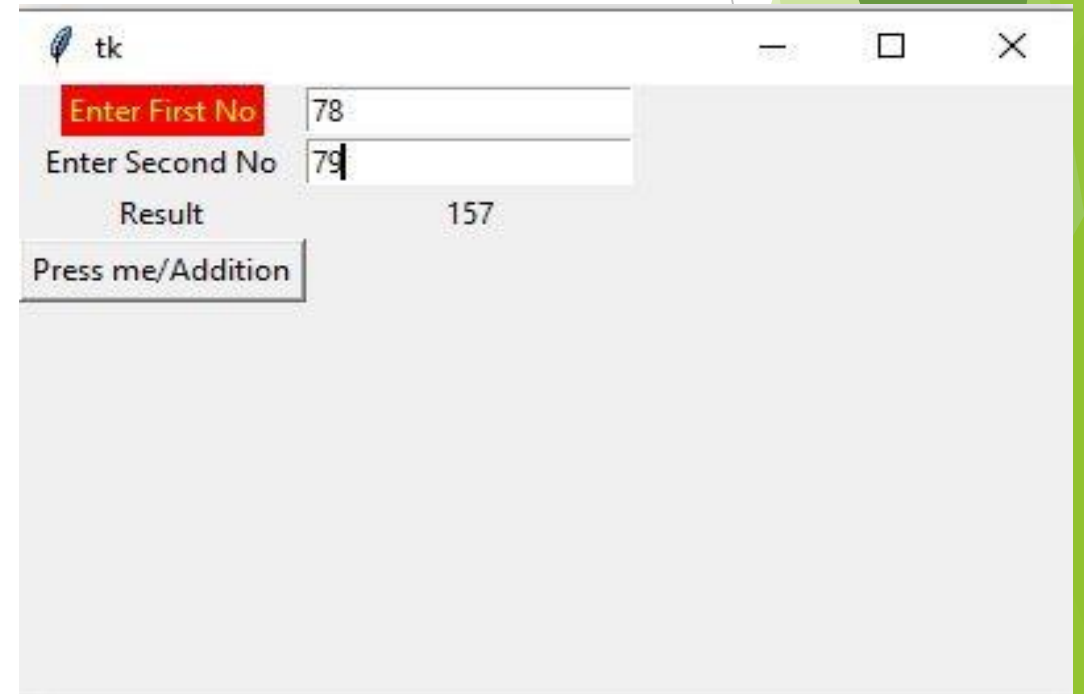
```
Label(top,textvariable=res).grid(row=2,column=1)
```

```
e1=Entry(top)  
e1.grid(row=0,column=1)
```

```
e2=Entry(top)  
e2.grid(row=1,column=1)
```

```
Button(top,text="Press me/Addition",command=add).grid(row=3,column=0)
```

```
top.mainloop()
```



Canvas in Tkinter

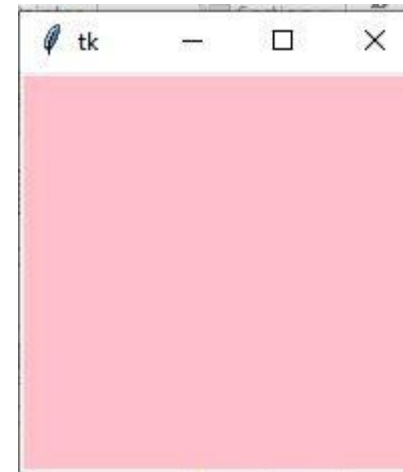
- ▶ The canvas widget is used to add the structured graphics to the python application.
- ▶ It is used to draw the graph and plots to the python application.

Syntax

```
w = canvas(parent, options)
```

Example

```
from tkinter import *  
  
top = Tk()  
  
top.geometry("200x200")  
  
#creating a simple canvas  
c = Canvas(top,bg = "pink",height = "200")  
  
c.pack()  
  
top.mainloop()
```



Drawing Objects inside Canvas

- ▶ **arc** – Creates an arc item, which can be a chord, a pieslice or a simple arc.
 - ▶ `coord = 10, 50, 240, 210 arc = canvas.create_arc(coord, start=0, extent=150, fill="blue")`
- ▶ **image** – Creates an image item, which can be an instance of either the `BitmapImage` or the `PhotoImage` classes.
 - ▶ `filename = PhotoImage(file = "sunshine.gif") image = canvas.create_image(50, 50, anchor=NE, image=filename)`
- ▶ **line** – Creates a line item.
 - ▶ `line = canvas.create_line(x0, y0, x1, y1, ..., xn, yn, options)`
- ▶ **oval** – Creates a circle or an ellipse at the given coordinates. It takes two pairs of coordinates; the top left and bottom right corners of the bounding rectangle for the oval.
 - ▶ `oval = canvas.create_oval(x0, y0, x1, y1, options)`
 - ▶ `r = canvas.create_rectangle(x0, y0, x1, y1, options)`
- ▶ **polygon** – Creates a polygon item that must have at least three vertices.
 - ▶ `oval = canvas.create_polygon(x0, y0, x1, y1,...xn, yn, options)`

Example

```
from tkinter import *
```

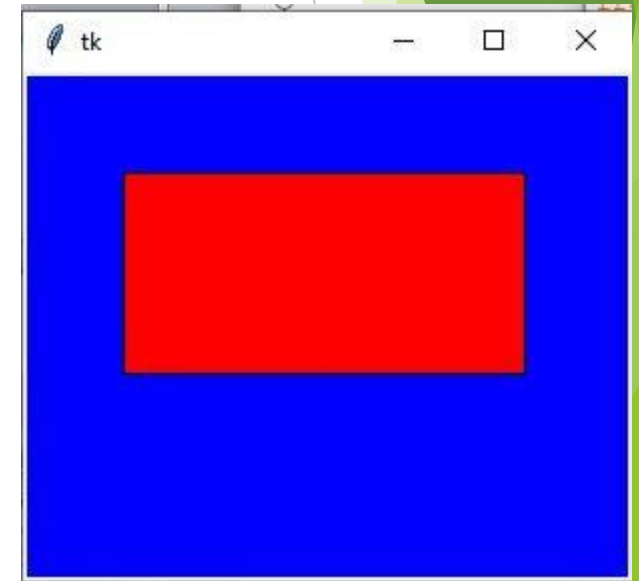
```
top = Tk()
```

```
C = Canvas(top, bg="blue", height=250, width=300)
```

```
C.create_rectangle(50,150,250,50,fill="red")
```

```
C.pack()
```

```
top.mainloop()
```



Radiobutton

- ▶ The Radiobutton widget is used to implement one-of-many selection in the python application. It shows multiple choices to the user out of which, the user can select only one out of them.

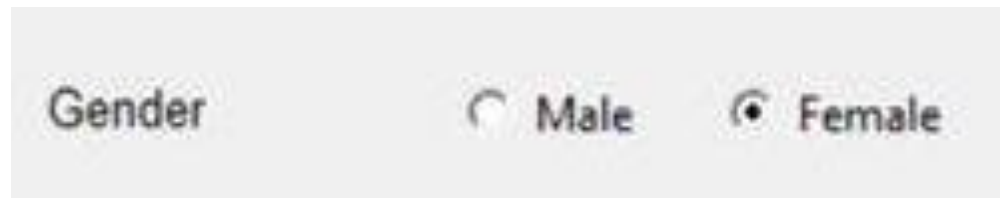
Syntax

```
w = Radiobutton(top, options)
```

```
var=IntVar()
```

```
Radiobutton(root,text="Male", variable= var, value=1).place(x=235,y=230)
```

```
Radiobutton(root,text="Female", variable= var, value=2).place(x=290,y=230)
```



Checkbox

- ▶ The Checkbox can contain the text or images. The Checkbox is mostly used to provide many choices to the user among which, the user needs to choose the one. It generally implements many of many selections.

Syntax

```
w = checkbox(master, options)
```

```
var1=IntVar()
```

```
Checkbox(root,text="English", variable=var1).place(x=230,y=330)
```

```
var2=IntVar()
```

```
Checkbox(root,text="German", variable=var2).place(x=290,y=330)
```



Language ☒ English ☐ German

OptionMenu

- ▶ OptionMenu is similar to dropdown list box where user can expand and select one of the several available options.
- ▶ `from tkinter import *`
- ▶ `top = Tk()`
- ▶ `options=StringVar()`
- ▶ `options.set("One")`
- ▶ `om1=OptionMenu(top, options, "HTML","PHP", "MySQL", "Python")`
- ▶ `om1.grid(row=2,column=2)`
- ▶ `top.mainloop()`



LIST BOX

A list box is useful to display a list of items in a box so that the user can select 1 or more items. To create a list box, we have to create an object of Listbox class, as:

```
lb = Listbox(f, font="Arial 12 bold", fg='blue', bg='yellow', height=8, width=24, activestyle='underline',  
selectmode=MULTIPLE)
```

The option 'selectmode' may take any of the following values:

- **BROWSE:** Normally, we can select one item (or line) out of a list box. If we click on an item and then drag to a different item, the selection will follow the mouse. This is the default value of 'selectmode' option.
- **SINGLE:** This represents that we can select only one item(or line) from all available list of items.
- **MULTIPLE:** We can select 1 or more number of items at once by clicking on the items. If an item is already selected, clicking second time on the item will un-select it.
- **EXTENDED:** We can select any adjacent group of items at once by clicking on the first item and dragging to the last item.

Once the list box is created, we should insert items into the list box using insert() method as:

```
lb.insert(0, 'Stanford University')  
lb.insert(1, 'Oxford University')
```

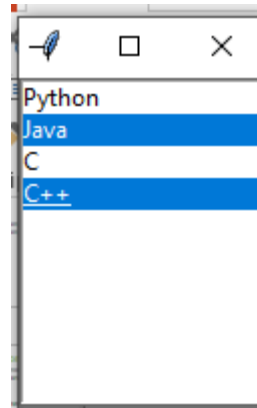
```
from tkinter import *
```

```
root = Tk()  
root.title("Simple Listbox")
```

```
# Create Listbox  
lb = Listbox(root,selectmode=MULTIPLE)  
lb.pack()
```

```
# Add items  
lb.insert(1, "Python")  
lb.insert(2, "Java")  
lb.insert(3, "C")  
lb.insert(4, "C++")
```

```
root.mainloop()
```



SPINBOX

- **Spinbox** is a GUI control in **Tkinter** that allows the user to **select a value by clicking up/down arrows** or by typing the value.
- It is mainly used when the input must be selected from a **fixed range of values** (like numbers, days, years, etc.).

✦ Definition

A **Spinbox** is a widget used to **choose one value from a sequence of values**.

✦ Why Spinbox is used?

Spinbox is used when:

- user must select only **limited values**
- we want to **avoid wrong input**
- input should be **easy and fast**

Example:

- Age (1 to 100)
- Month number (1 to 12)

✈ General Syntax

`Spinbox(parent, from_=start, to=end)`

Option

`from_`

`to`

`width`

`textvariable`

Meaning

Starting value

Ending value

Size of the box

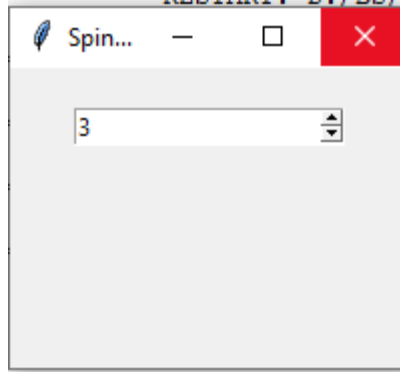
Variable to store value

```
from tkinter import *
```

```
root = Tk()
root.title("Spinbox Example")
root.geometry("200x150")
```

```
# Create Spinbox
sp = Spinbox(root, from_=1, to=10)
sp.pack(pady=20)
```

```
root.mainloop()
```



You can try with listbox, spinbox, frame etc.,

Lets try to create Registration form now

```
from tkinter import *
```

```
root = Tk()
```

```
#Providing Geometry to the form  
root.geometry("500x500")
```

```
#Providing title to the form  
root.title('Registration form')
```

```
label_0 =Label(root,text="Registration form", width=20,font=("bold",20))  
label_0.place(x=90,y=60)
```

```
label_1 =Label(root,text="FullName", width=20,font=("bold",10))  
label_1.place(x=80,y=130)
```

```
entry_1=Entry(root)  
entry_1.place(x=240,y=130)
```

```
label_3 =Label(root,text="Email", width=20,font=("bold",10))  
label_3.place(x=68,y=180)
```

```
entry_3=Entry(root)  
entry_3.place(x=240,y=180)
```

```
label_4 =Label(root,text="Gender", width=20,font=("bold",10))  
label_4.place(x=70,y=230)
```

Registration form

FullName

Email

Gender ☐ Male ☐ Female

Country

Language ☐ English ☐ German

```
var=IntVar()
```

```
Radiobutton(root,text="Male",padx= 5, variable= var, value=1).place(x=235,y=230)
```

```
Radiobutton(root,text="Female",padx= 20, variable= var, value=2).place(x=290,y=230)
```

```
label_5=Label(root,text="Country",width=20,font=("bold",10))
```

```
label_5.place(x=70,y=280)
```

```
c=StringVar()
```

```
droplist=OptionMenu(root,c, 'India' , 'US' , 'UK' , 'Germany' , 'Austria')
```

```
c.set('Select your Country')
```

```
droplist.place(x=240,y=280)
```

```
label_6=Label(root,text="Language",width=20,font=('bold',10))
```

```
label_6.place(x=75,y=330)
```

```
var1=IntVar()
```

```
Checkbutton(root,text="English", variable=var1).place(x=230,y=330)
```

```
var2=IntVar()
```

```
Checkbutton(root,text="German", variable=var2).place(x=290,y=330)
```

The image shows a Tkinter window titled "Registration form" with a light gray background. The form contains the following elements:

- FullName:** A text input field.
- Email:** A text input field.
- Gender:** Two radio buttons labeled "Male" and "Female".
- Country:** A dropdown menu with the text "Select your Country".
- Language:** Two checkboxes labeled "English" and "German".
- Submit:** A black button with the text "Submit" in white.

```
# ----- Submit button function -----
```

```
def submit():
```

```
    name = entry_1.get()
    email = entry_3.get()
```

```
    if var.get() == 1:
        gender = "Male"
    elif var.get() == 2:
        gender = "Female"
    else:
        gender = "Not selected"
```

```
    country = c.get()
```

```
    lang = []
    if var1.get() == 1:
        lang.append("English")
    if var2.get() == 1:
        lang.append("German")
```

```
    languages = ", ".join(lang)
```

```
    messagebox.showinfo(
        "Submitted Details",
        "Name : " + name +
        "\nEmail : " + email +
        "\nGender : " + gender +
        "\nCountry : " + country +
        "\nLanguage : " + languages
    )
```

```
Button(root, text='Submit' , width=20,bg="black",fg='white',command=submit).place(x=180,y=380)
root.mainloop()
```

The image shows a Python Tkinter application window titled "Registration form". The form contains the following fields and controls:

- FullName**: A text entry field containing "AJAY".
- Email**: A text entry field containing "arasuajay@gmail.com".
- Gender**: Two radio buttons, "Male" (selected) and "Female".
- Country**: A dropdown menu showing "India".
- Language**: Two checkboxes, "English" (checked) and "German".
- Submit**: A black button with white text.

Overlaid on the bottom right of the registration form is a smaller dialog box titled "Submitted Details". It contains an information icon and the following text:

Name : AJAY
Email : arasuajay@gmail.com
Gender : Male
Country : India
Language : English

An "OK" button is located at the bottom right of the "Submitted Details" dialog.

THANK YOU